

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

Members:

YADAV Anshuman Krishna (exercise 1)

MAHALINGAM Nithees (exercise 2)

SARAVANAN Arun Prasath (exercise 3)

Github Link:

https://github.com/anshuman-krishna/advanced-algorithms-and-programming/tree/LAB_05_BasicsOfTrees

Exercise 1: YADAV Anshuman Krishna

Binary Tree representation of Content Categories

1. CategoryNode Structure

```
STRUCTURE CategoryNode
category_id : String
name : String
post_count : Integer
left : CategoryNode
right : CategoryNode
parent : CategoryNode
END STRUCTURE
```

Time Complexity : $O(1)$

Space Complexity : $O(1)$

2. Tree Metric Calculations

(note: for tree-metrics, i had to google some parts since i missed the course class on this topic due to the symposium (simuvaction), however, i also went through the lecture notes to understand it better, the googled references for the books i used are:

1. Data structures and Algorithms by Granville Barnett and Luca Del Tongo)
2. Competitive programming by Steven Halim and Felix Halim
thanks.

A. calculate_height

```
FUNCTION calculate_height(node) -> Integer
IF node = NULL THEN
RETURN 0 END IF
```

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

```
left_height <- calculate_height(node.left)
right_height <- calculate_height(node.right)
RETURN Max(left_height, right_height) + 1
```

Time Complexity : $O(N)$

Space Complexity : $O(H)$, H is height of the tree, i.e max no. of recursive calls here

B. calculate_node_height

```
FUNCTION calculate_node_height(node, target_id) -> Integer
target_node <- find_category(target_id, node)
```

```
IF target_node = NULL THEN
RETURN 0 END IF
RETURN calculate_height(target_node)
```

Time Complexity : $O(N)$ for node search, $O(N)$ for height, total $O(N)$

Space Complexity : $O(H)$, recursive call stack

C. count_node

```
FUNCTION count_nodes(node) -> Integer
IF node = NULL THEN
RETURN 0 END IF

RETURN 1 + count_nodes(node.left) + count_nodes(node.right)
```

Time Complexity : $O(N)$

Space Complexity : $O(H)$, call stack

D. count_leaves

```
FUNCTION count_leaves(node) -> Integer
IF node = NULL THEN
RETURN 0 END IF

IF node.left = NULL AND node.right = NULL THEN
RETURN 1 END IF
RETURN count_leaves((node.left) + (node.right))
```

Time Complexity : $O(N)$

Space Complexity : $O(H)$, call stack

E. is_balanced

```
FUNCTION is_balanced(node) -> Boolean
IF node = NULL THEN
RETURN TRUE END IF

left_h <- calculate_height(node.left)
right_h <- calculate_height(node.right)
diff <- Abs(left_h - right_h)

IF diff <= 1 AND is_balanced(node.left) = TRUE AND is_balanced(node.right) = TRUE
THEN RETURN TRUE
END IF RETURN FALSE
```

Time Complexity : $O(N^2)$, because we recalculate the height as we traverse down

Space Complexity : $O(H)$, call stack

3. Tree Property Verification

A. is_full_binary_tree

```
FUNCTION is_full_binary_tree(node) -> Boolean
IF node = NULL THEN
RETURN TRUE END IF

IF node.left = NULL AND node.right = NULL THEN
RETURN TRUE END IF

IF node.left != NULL AND node.right != NULL THEN
RETURN is_full_binary_tree(node.left) AND is_full_binary_tree(node.right)
END IF RETURN FALSE
```

Time Complexity : $O(N)$

Space Complexity : $O(H)$,

Because we only check if each node has 0 or 2 children.

B. is_perfect_binary_tree

```
FUNCTION is_perfect_binary_tree(node) -> Boolean
IF is_full_binary_tree(node) = TRUE AND is_balanced(node) = TRUE THEN
RETURN TRUE END IF
RETURN FALSE
```

Time Complexity : $O(n^2)$ because of the “is_balanced” function we used.

Space Complexity : $O(H)$

C. is_complete_binary_tree

Time Complexity :

Space Complexity :

...

will include in T24_LAB5_BFM

...

4. Category Search and Navigation

A. find_category

FUNCTION find_category(category_id, node) -> CategoryNode

IF node = NULL THEN

RETURN NULL END IF

IF node.category_id = category_id THEN

RETURN node END IF

left_search <- find_category(category_id, node.left)

IF left_search != NULL THEN

RETURN left_search END IF

RETURN find_category(category_id, node.right)

Time Complexity : $O(N)$

Space Complexity : $O(H)$ because call stack; H is height of the tree

B. find_path_to_root

FUNCTION find_path_to_root(category_id, node) -> List

target <- find_category(category_id, node)

path <- empty list

WHILE target != NULL DO

path.Add(target.name)

target <- target.parent

END WHILE

RETURN path

Time Complexity : $O(N)$ to find the node and $O(H)$ to trace the parent pointers

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

Space Complexity : $O(H)$ to store path array

C. lowest_common_ancestor

(had to google this one too, still didn't understand)

```
FUNCTION lowest_common_ancestor(id1, id2, node) -> CategoryNode
```

```
IF node = NULL THEN
```

```
RETURN NULL END IF
```

```
IF node.category_id = ...
```

```
...
```

```
will hopefully include in T24_LAB5_BFM
```

```
...
```

Complexity Analysis Questions:

1. What is the time complexity of calculating tree height? Prove it.?

Answer:

The time complexity is $O(N)$. because to find the height of the tree, the algorithm shall must visit every single node exactly once to see how deep the branches go till.

So if we have a tree with N nodes, the recurrence relation is:

$T(N) = T(\text{Left}) + T(\text{Right}) + O(1)$. Since the sum of left and right nodes is $N-1$, the total operations scale linearly as N .

2. For a perfectly balanced binary tree with n nodes, what is the maximum height?

Answer:

The maximum height would approximately be $\log_2(n)$, because every level is completely filled and the number of nodes doubles at each level.

3. In a social network with millions of categories, why would balance matter?

Answer:

In such scenario, the balance would be dictating the search speed. If a tree of

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

millions of categories is unbalanced (every node has only a right child), it can technically degrade into being a linked list. Searching for a particular node would take $O(N)$, hence further forcing the server to check million items one-by-one. If the tree is balanced, the search space will be cut in half at each step, hence making the search time $O(\log N)$, now that might not look a huge gap for smaller values but with millions of categories, the operation which couldve been milliseconds would most probably freeze the entire application.

Exercise 2: MAHALINGAM Nithees

Tree Traversals for Content Processing

1. class Node:

```
    name
    posts
    children
    total_posts
```

Part A : In-order Traversal (Left -> Root -> Right)

in_order_collect(node):

```
function in_order(node):
    if node == null:
        return []
    result = []
    n = length(node.children)
    mid = floor(n / 2)
    for i = 0 to mid - 1:
        result += in_order(node.children[i])
    result.append(node.name)
    for i = mid to n - 1:
        result += in_order(node.children[i])
    return result
```

in_order_accumulate_posts(node):

```
function in_order_accumulate(node):
    if node == null:
        return 0
    total = 0
```

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

```
n = length(node.children)
mid = floor(n / 2)
for i = 0 to mid - 1:
    total += in_order_accumulate(node.children[i])
total += node.posts
for i = mid to n - 1:
    total += in_order_accumulate(node.children[i])
return total
```

in_order_find_kth(k, node):

```
function in_order_find_kth(node, k):
    result = in_order(node)
    if k <= 0 or k > length(result):
        return null
    return result[k - 1]
```

Part B : Pre-order Traversal (Root -> Left -> Right)

pre_order_export(node):

```
function pre_order(node):
    if node == null:
        return
    print(node.name)
    for child in node.children:
        pre_order(child)
```

pre_order_copy(node):

```
function pre_order_copy(node):
    if node == null:
        return null
    new_node = new Node
    new_node.name = node.name
    new_node.posts = node.posts
    new_node.children = []
    for child in node.children:
        copied_child = pre_order_copy(child)
        new_node.children.append(copied_child)
    return new_node
```

pre_order_serialize(node):

```
function pre_order_serialize(node, depth):
    if node == null:
```

```
    return ""
output = ""
for i = 0 to depth - 1:
    output += " "
output += node.name + "\n"
for child in node.children:
    output += pre_order_serialize(child, depth + 1)
return output
```

Part C : Post-order Traversal (Left -> Right -> Root)

post_order_total_posts(node):

```
function post_order_total_posts(node):
    if node == null:
        return 0
    total = node.posts
    for child in node.children:
        total += post_order_total_posts(child)
    node.total_posts = total
    return total
```

post_order_average_depth(node):

```
function post_order_avg_depth(node, depth):
    if node == null:
        return (0, 0)
    if length(node.children) == 0:
        return (depth, 1)
    total_depth = 0
    total_count = 0
    for child in node.children:
        (d, c) = post_order_avg_depth(child, depth + 1)
        total_depth += d
        total_count += c
    return (total_depth, total_count)
function compute_average_depth(root):
    (sum_depth, count) = post_order_avg_depth(root, 0)
    if count == 0:
        return 0
    return sum_depth / count
```

post_order_collect_leaves(node):

```
function post_order_collect_leaves(node):
    if node == null:
```



```
    return []
if length(node.children) == 0:
    return [node.name]
leaves = []
for child in node.children:
    leaves += post_order_collect_leaves(child)
return leaves
```

2. Traversal Based Analytics:

find_most_popular_category(node):

```
function find_most_popular_category(node):
    if node == null:
        return (null, -infinity)
    if length(node.children) == 0:
        return (node.name, node.posts)
    best_name = null
    best_posts = -infinity
    for child in node.children:
        (name, posts) = find_most_popular_category(child)
        if posts > best_posts:
            best_posts = posts
            best_name = name
    return (best_name, best_posts)
```

find_most_popular_category(node):

```
function category_with_most_subcategories(node):
    if node == null:
        return (null, -1)
    max_node = node
    max_count = length(node.children)
    for child in node.children:
        (child_name, child_count) = category_with_most_subcategories(child)
        if child_count > max_count:
            max_count = child_count
            max_node = child_name
    return (max_node, max_count)
```

Complexity Analysis Questions:

1. What is the time and space complexity of each traversal?

Answer:

Time Complexity = $O(n)$

Reason: Every node is visited exactly once.

Space Complexity:

Recursive:

$O(h)$ (call stack)

- h = height of tree
- Worst case (skewed tree): **$O(n)$**
- Balanced tree: **$O(\log n)$**

Additional Space:

If storing results (like lists): $O(n)$

2. When would you use pre-order vs. post-order for aggregating metrics?

Answer:

Pre-order (Root $\rightarrow$ Children):

Use when:

- We need to copy the tree
- We need to serialize/export structure
- We process parent before children

Post-order (Children $\rightarrow$ Root):

Use when:

- We need to aggregate data from children
- We compute values like:
 - totals
 - averages
 - depths

3. How would you implement an iterative version of these traversals using a stack?

Answer:

...///BFM

4. For a social network's category export feature, which traversal is most appropriate and why?

Answer:

Post Order Traversal is the best choice. Because:

Social platforms often compute:

- total engagement
- aggregated metrics
- popularity scores

These depend on children first, then parent

Exercise 3: SARAVANAN Arun Prasath

Generalized Trees (N-ary Trees) and Representations

1. Create a GeneralizedCategoryNode structure

```
STRUCT Node
category_id
name
post_count
children = empty list
parent = null
END STRUCT
```

2. Conversion Between Representations

Part-A: Convert Binary Tree - Generalized Tree

```
FUNCTION binary_to_generalized(binary_node, parent = null)
IF binary_node IS null
RETURN null
END IF
CREATE new_node
```

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

```
new_node.id = binary_node.id
new_node.name = binary_node.name
new_node.parent = parent
Convert left subtree - children
child = binary_node.left
WHILE child IS NOT null
  converted_child = binary_to_generalized(child, new_node)
  ADD converted_child TO new_node.children
  move to next sibling (right pointer)
  child = child.right
END WHILE
RETURN new_node
END FUNCTION
```

Part-B: Convert Generalized Tree → Binary Tree

```
FUNCTION generalized_to_binary(gen_node)
  IF gen_node IS null
    RETURN null
  END IF
  CREATE binary_node
  binary_node.id = gen_node.id
  binary_node.name = gen_node.name

  IF gen_node.children IS NOT empty

    binary_node.left = generalized_to_binary(gen_node.children[0])
    current = binary_node.left

    FOR i FROM 1 TO length(gen_node.children) - 1
      current.right = generalized_to_binary(gen_node.children[i])
      current = current.right
    END FOR
  END IF
  RETURN binary_node
END FUNCTION
```

3. Generalized Tree Traversals

Pre-order:

```
FUNCTION pre_order_generalized(node)
  IF node IS null
    RETURN
```

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

```
END IF
PRINT node.name
FOR each child IN node.children
  pre_order_generalized(child)
END FOR
END FUNCTION
```

Post-order:

```
FUNCTION post_order_generalized(node)
  IF node IS null
    RETURN
  END IF
  FOR each child IN node.children
    post_order_generalized(child)
  END FOR
  PRINT node.name
END FUNCTION
```

Level-order (BFS)

```
FUNCTION level_order_generalized(root)
  CREATE queue
  ADD root TO queue
  WHILE queue IS NOT empty
    current = REMOVE FROM queue
    PRINT current.name
    FOR each child IN current.children
      ADD child TO queue
    END FOR
  END WHILE
END FUNCTION
```

Calculate Fan-out (max children)

```
FUNCTION calculate_fan_out(node)
  IF node IS null
    RETURN 0
  END IF
  max_children = size(node.children)
  FOR each child IN node.children
    child_max = calculate_fan_out(child)
    IF child_max > max_children
      max_children = child_max
    END IF
  END FOR
END FUNCTION
```

```
END FOR
RETURN max_children
END FUNCTION
```

4. Tree Metrics

Height:

```
FUNCTION calculate_height_generalized(node)
IF node IS null
RETURN 0
END IF
max_height = 0
FOR each child IN node.children
h = calculate_height_generalized(child)
IF h > max_height
max_height = h
END IF
END FOR
RETURN max_height + 1
END FUNCTION
```

Count Nodes

```
FUNCTION count_nodes_generalized(node)
IF node IS null
RETURN 0
END IF
total = 1
FOR each child IN node.children
total = total + count_nodes_generalized(child)
END FOR
RETURN total
END FUNCTION
```

Count Leaves

```
FUNCTION count_leaves_generalized(node)
IF node IS null
RETURN 0
END IF
IF node.children IS empty
RETURN 1
END IF
total = 0
```

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

```
FOR each child IN node.children
total = total + count_leaves_generalized(child)
END FOR
RETURN total
END FUNCTION
```

Branching Factor (average children)

```
FUNCTION calculate_branching_factor(root)
total_children = 0
non_leaf_nodes = 0
FUNCTION helper(node)
IF node IS null
RETURN
END IF
IF node.children IS NOT empty
total_children += size(node.children)
non_leaf_nodes += 1
END IF
FOR each child IN node.children
helper(child)
END FOR
END FUNCTION
helper(root)
RETURN total_children / non_leaf_nodes
END FUNCTION
```

Complexity Analysis Questions:

1.. What are the advantages and disadvantages of each representation (binary vs. generalized)?

Answer:

Tree Types	Advantages	Disadvantages
Binary Tree	-Simple and easy to implement -Requires less memory (only 2 pointers) -Easy traversal	-Can have only 2 children - Not flexible for real-world data - Not suitable for complex hierarchies

II.2415 Advanced Algorithms and Programming

Lab 5: Basics of Tree Structures: Team T24

Generalized Tree	- Can have any number of children - Very flexible structure - Best for real-world applications (like categories)	- More complex to implement - Uses extra memory (list of children) - Traversal is slightly harder
-------------------------	--	---

2. For a generalized tree with n nodes and branching factor b , what is the space complexity of each representation?

Answer:

3. How does traversal complexity differ between binary and generalized Representations?

Answer:

4. In a social network with millions of categories, which representation would you choose and why?

Answer:

5. Explain the "first child, next sibling" transformation and its time complexity

Answer:

Answer:

Final Integration Question

to be continued in T24_LAB4_BFM